

从认知科学到 智能体记忆

先看清人脑怎么记，再为 Agent 造一套完整的记忆系统。

主讲人：柠檬酸

Agent 也要记性

前面章节里 Agent 能调用工具、能规划任务，但它有一个致命的短板——每次对话结束，它就什么都忘了。这一节我们先看人类怎么记忆，再把它翻译成可工程化的智能体记忆系统。



8.1.1	8.1.2	8.1.3	8.1.4
认知科学启发	为何需要记忆与 RAG	系统架构设计	学习目标与快速体验
人类记忆如何分层，启发 Agent 设计	LLM 两大根本性局限	记忆系统 + RAG 系统双轨	30 秒上手 + 项目结构

人类记忆的层次结构

认知心理学把记忆分成三层，每一层都有自己的容量、寿命、用途



三类核心 + 程序性记忆 → 共四类长期记忆子类型

LLM 的两个短板

借鉴人类记忆，我们可以看清 LLM 智能体为什么必须外挂记忆与检索

LIMITATION 1

对话状态的遗忘

LLM 本身是无状态的——每一次请求都是独立的计算，不会自动"记住"上一轮内容。

后果：上下文丢失 · 个性化缺失 · 学习能力受限 · 前后矛盾

LIMITATION 2

内置知识的局限

LLM 的知识是静态、有限的，全部来自训练数据——天然带截止时间和覆盖面。

后果：时效性差 · 专业深度不足 · 容易幻觉 · 缺少来源

无状态：模型不会自动"记住"

每次请求都是独立的——这直接导致上下文丢失、个性化缺失、学习能力受限

```
# 第七章的 Agent 使用方式
from hello_agents import SimpleAgent, HelloAgentsLLM
agent = SimpleAgent(name="学习助手", llm=HelloAgentsLLM())

# 第一次对话
response1 = agent.run("我叫张三，正在学习Python，目前掌握了基础语法")
# 第二次对话（新的会话）
response2 = agent.run("你还记得我的学习进度吗？")
# → "抱歉，我不知道您的学习进度..."
```

要解决这个短板，HelloAgents 引入记忆系统

静态知识： 截止时间 + 覆盖面

时效性 训练截止 大模型的训练数据有截止点，无法获取最新信息——昨天的新闻它听都没听过 用实时检索补	深度 专业领域 通用模型在垂直领域（医学、法律、论文）深度知识可能不足——样样通、样样松 用专业语料补
事实 幻觉风险 模型可能编造看似合理但并不存在的事实——"一本正经地说错话" 用检索验证补	来源 可解释性 直接问 LLM 它答不出"为什么这么说"，只能凭训练权重编一条最像的 用引用追溯补

RAG：检索增强生成

为了克服静态知识的局限，RAG 应运而生——生成答案之前，先去外部知识库把相关资料找出来



BEFORE · 闭卷答题

凭训练权重答

LLM 只靠训练时学到的知识——答得自信，但时效不准、深度不够、容易幻觉，且没有出处。

比喻 · 一个没带资料就上场的学生

AFTER · 开卷作答

检索 + 增强 + 生成

先去外部知识库查最新资料，再把资料融进 Prompt，最后驱动 LLM 作答——兼具准确性与透明度。

比喻 · 翻书查目录 → 摊资料上桌 → 参考答案动笔

HelloAgents 的双轨设计

借鉴人类记忆层次 + 兼顾工程实现——记忆与 RAG 设计为两个独立工具

TOOL 1 · MEMORY

memory_tool

存储和维护对话过程中的交互信息——智能体自己的经历。

职责 · 对话历史 · 用户偏好 · 长期指令 · 跨会话一致性

TOOL 2 · KNOWLEDGE

rag_tool

从用户提供的知识库中检索相关信息作为上下文，并可将重要检索结果自动存储到记忆系统。

职责 · 外部文档检索 · 专业知识问答 · 答案引用与可解释

记忆系统与 RAG 系统的代码结构

两套系统都采用分层架构，从基础设施到上层工具逐级可插拔

MEMORY · 四层架构

HelloAgents 记忆系统

- 基础设施层 MemoryManager · MemoryItem · MemoryConfig · BaseMemory
- 记忆类型层 WorkingMemory · EpisodicMemory · SemanticMemory · PerceptualMemory
- 存储后端层 QdrantVectorStore · Neo4jGraphStore · SQLiteDocumentStore
- 嵌入服务层 DashScope · LocalTransformer · TFIDF

工程模式 · 同模块同接口，存储可替换

RAG · 端到端管道

HelloAgents RAG 系统

- 文档处理层 DocumentProcessor · Document · Pipeline
- 嵌入表示层 复用记忆系统的 EmbeddingService
- 向量存储层 QdrantVectorStore (命名空间隔离)
- 智能问答层 多策略检索 · 上下文构建 · LLM 增强生成

设计要点 · 嵌入服务统一，避免重复实现

项目代码结构 + 学习路径

遵循第七章"工具化"原则，记忆与 RAG 都封装为标准 Tool，不创建新的 Agent 类

```
hello-agents/  
└─ hello_agents/  
    │   └─ memory/                                # 记忆系统模块  
    │       │   └─ manager.py · embedding.py · base.py  
    │       │   └─ types/    working.py · episodic.py · semantic.py · perceptual.py  
    │       │   └─ storage/  qdrant_store.py · neo4j_store.py · document_store.py  
    │       │   └─ rag/      pipeline.py · document.py  
    └─ tools/builtin/  
        │   └─ memory_tool.py    # Agent 记忆能力  
        └─ rag_tool.py           # 智能问答能力
```

两条学习路径：① 体验式——pip 安装直接跑示例 ② 深度式——从零实现每个组件并跑测试

快速体验：让 Agent 拥有记忆与知识

pip 安装 → 配置 .env → 注册两个 Tool 到 SimpleAgent——三步上线

```
# 配置好同级文件夹下 .env 中的大模型 API
from hello_agents import SimpleAgent, HelloAgentsLLM, ToolRegistry
from hello_agents.tools import MemoryTool, RAGTool

llm = HelloAgentsLLM()
agent = SimpleAgent(name="智能助手", llm=llm, system_prompt="...")

tool_registry = ToolRegistry()
tool_registry.register_tool(MemoryTool(user_id="user123"))
tool_registry.register_tool(RAGTool(knowledge_base_path="./knowledge_base"))
agent.tool_registry = tool_registry

# 开始对话—从此 Agent 既能"记住"也能"查资料"
response = agent.run("你好! 请记住我叫张三, 我是一名 Python 开发者")
```

pip install

pip install "hello-agents[all]==0.2.0" + spacy 中英文模型

.env 三件套

QDRANT_URL/API_KEY + NEO4J_URI/PASSWORD +
EMBED_MODEL_TYPE=dashscope

两个核心方法

memory_tool.execute("add"/"search") ·
rag_tool.execute("add_text"/"search")

CLOSING

从认知科学出发，
到 HelloAgents 的记忆 + RAG 双轨设计——
让 Agent 真正拥有记忆与持续学习能力。